

USD / EUR

0.87207

USD / GBP

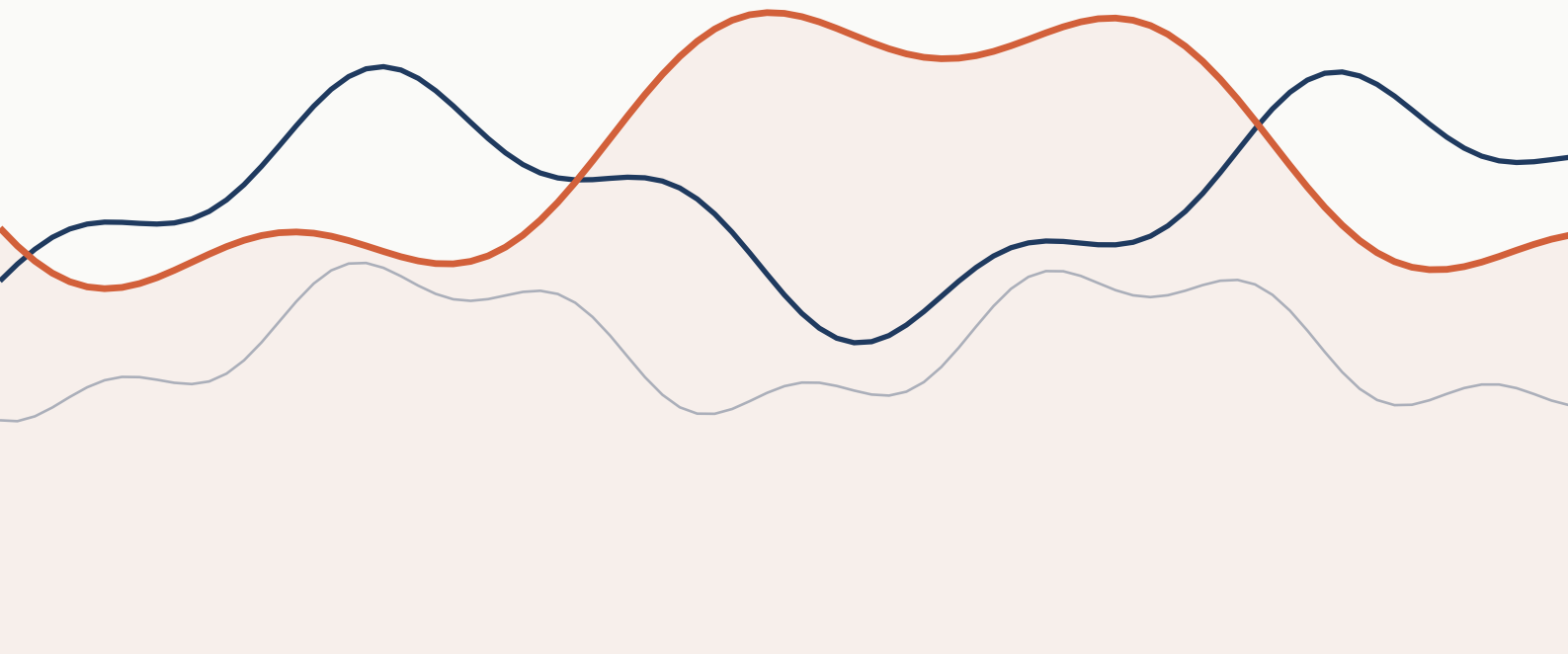
0.75567

USD / AUD

1.4257

USD / JPY

161.23



DATA ENGINEERING & ANALYTICS

Crestview Finance.

Market Risk Intelligence Feed: daily FX rates from a public API into the warehouse.

A Microsoft Fabric case study in warehouse-native API ingestion: a Web Activity, T-SQL JSON shredding with OPENJSON, and a stored procedure that lands one clean row per currency per day.

Five stops between the web and the warehouse.

A lender with a \$4.2B international book. Exchange rates copied from public websites once a month. A 5% euro move worth \$55M that nobody sees for weeks. This is the brief.

-
- 01

The Brief
The Head of Risk, three problems in numbers and four options weighed.

p. 03
-
- 02

The Source
The Frankfurter API and how one nested payload becomes four rows.

p. 04
-
- 03

The Build
The daily pipeline, the target table and four mandates.

p. 05
-
- 04

Lab & Acceptance
Four class deliverables, three assignment tasks, one bonus, and the checks.

p. 06
-
- 05

Discussion
Five design questions: engine, payload hand-off, parsing, reruns, weekends.

p. 07
-

SECTION 01 · THE BRIEF

Operationalise the rate feed. Before the next volatility event.

Crestview Finance lends across North America, the Eurozone and APAC, and reports every exposure in US dollars. The exchange rates behind those numbers are copied from public websites into Excel once a month. The Head of Risk wants a daily, automated feed in the Fabric warehouse.

“I cannot sign off on international risk reports built on a month-old conversion rate. If the Euro drops five percent overnight, I need to see that impact on capital reserves the next morning, not next month.”

HEAD OF RISK, CRESTVIEW FINANCE

Monthly

RATE REFRESH TODAY

Stale rates

Rates are copied from public websites once a month. Every report in between runs on a conversion that may be four weeks old.

\$55M

USD SWING FROM A 5% EUR MOVE

Exposure nobody sees

\$1.1B of the \$4.2B international book is in euros. A 5% overnight move shifts its dollar value by about \$55M, and a monthly rate hides it for weeks.

16h

ANALYST HOURS PER MONTH

Manual transcription

Two risk analysts spend about eight hours a month each finding, checking and typing rates. Decimal slips surface at quarter-end, if at all.

FOUR OPTIONS, ONE RECOMMENDATION

Do nothing (baseline) REJECTED	FIXES Nothing to build.	LEAVES BROKEN Monthly rates, manual typing, and a \$55M swing unseen for weeks.
Excel Power Query from the web REJECTED	FIXES Rates refresh when an analyst opens the file.	LEAVES BROKEN Every workbook holds its own rates. No shared history, nothing an auditor can trace.
Licensed market-data feed REJECTED	FIXES Authoritative, intraday rates.	LEAVES BROKEN Licence cost for four currencies once a day, and it still needs a pipeline into the warehouse.
Fabric pipeline into the warehouse RECOMMENDED	FIXES Daily rates from the API, one table every team reads, fetch time recorded.	LEAVES BROKEN Rates arrive once a day after the ECB publishes, not intraday.

Only the warehouse feed gives every team the same rate for the same date, with a record of when it was fetched. The price is a daily rate, not an intraday one, and the Head of Risk accepts it: he needs yesterday's move by the morning.

SECTION 02 · THE SOURCE

One public API. One nested payload.

Frankfurter publishes the European Central Bank's reference rates as JSON. The feed needs four currencies against the US dollar: EUR, JPY, GBP and AUD.

EXHIBIT A · FRANKFURTER API

BASE URL

`https://api.frankfurter.dev/v1/`

Set once on the Web connection.

LATEST RATES

`latest?base=USD&symbols=EUR,JPY,GBP,AUD`

One day of rates. Used by the daily pipeline.

DATE RANGE

`YYYY-MM-DD..YYYY-MM-DD`

Many days in one call. Leave the end open to run to today.

PUBLISHING

ECB working days, about 16:00 CET

No new rates on weekends or TARGET holidays. Dates are stored in UTC.

EXHIBIT B · FROM PAYLOAD TO ROWS

```
{
  "amount": 1.0,
  "base": "USD",
  "date": "2026-06-19",
  "rates": {
    "AUD": 1.4257,
    "EUR": 0.87207,
    "GBP": 0.75567,
    "JPY": 161.23
  }
}
```

`$.date`

→ **RateDate**

`$.base`

→ **BaseCurrency**

`rates key`

→ **TargetCurrency**

`rates value`

→ **RateValue**

RATE DATE	BASE	TARGET	RATE VALUE
2026-06-19	USD	AUD	1.4257
2026-06-19	USD	EUR	0.87207
2026-06-19	USD	GBP	0.75567
2026-06-19	USD	JPY	161.23

SECTION 03 · THE BUILD

A Web Activity. A stored procedure. One table.

You are the Lead Data Engineer. Build the feed inside the Fabric warehouse, from the API call to the table the risk models read.

EXHIBIT C · THE DAILY PIPELINE

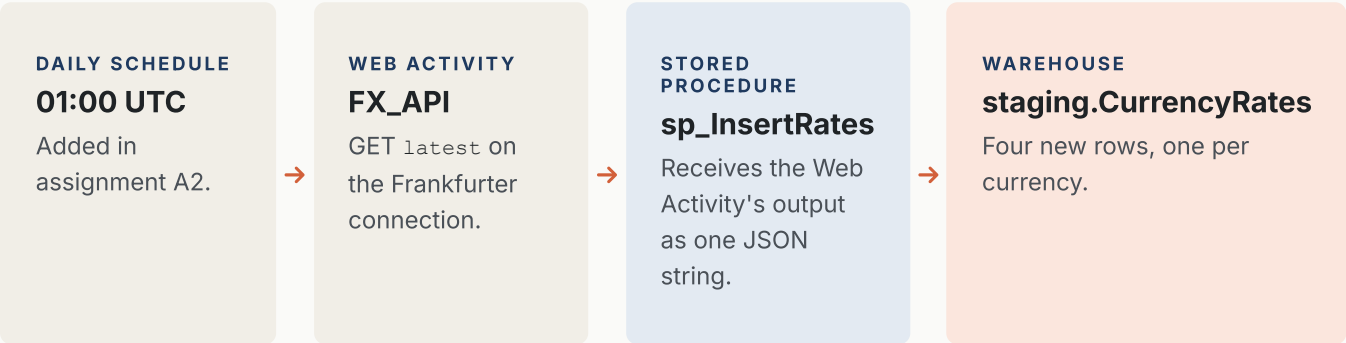


EXHIBIT D · TARGET SCHEMA

staging.CurrencyRates		TARGET TABLE
RateDate	DATE	KEY
BaseCurrency	CHAR(3)	ANCHOR
TargetCurrency	CHAR(3)	DIM
RateValue	DECIMAL(20,6)	MEASURE
FetchAtUtc	DATETIME2(6)	AUDIT

- BaseCurrency is always USD in this feed.
- FetchAtUtc is added in assignment A3.

FOUR MANDATES

- 01 Automation**

No manual rate entry. The pipeline fetches and loads rates on its own.

Met by: D3, A2
- 02 In-warehouse parsing**

The JSON is shredded in T-SQL inside the warehouse.

Met by: D2
- 03 History**

Every ECB business day since 1 October 2026 is in the table.

Met by: A1
- 04 Traceability**

Every row records when it was fetched, in UTC.

Met by: A3

SECTION 04 · LAB & ACCEPTANCE

Four in class. Three at home. One bonus.

Class builds the daily feed. The assignment adds history, a schedule and an audit trail. The bonus makes reruns safe. Each stage is signed off by its own check.

IN CLASS

4

deliverables

Built together in the session.

D1 Warehouse and table

The staging schema and `staging.CurrencyRates`, matching the target schema.

D2 `sp_InsertRates`

Shreds a single-day payload with OPENJSON. Tested first with a hard-coded JSON string.

D3 Daily pipeline

Web Activity calls `latest`, then hands its output to `sp_InsertRates`.

D4 Validation query

Rows per date, and the latest rate for each currency.

DONE WHEN

Rows land.

Four rows for the latest date, one per currency, matching the API to six decimals.

ASSIGNMENT

3

tasks

Done at home before the next session.

A1 Backfill

Every business day since 1 October 2026, in one API call, shredded into the same table.

A2 Schedule

The daily pipeline runs at 01:00 UTC.

A3 Audit column

`FetchAtUtc` added without dropping the table, filled on every new row.

DONE WHEN

History and audit.

One date for every ECB business day since 1 October 2026. `FetchAtUtc` set on every backfilled row.

BONUS

1

stretch

For those who finish early.

B1 Safe reruns

Loading the same dates twice leaves the row count unchanged.

DONE WHEN

The replay test.

Run the backfill twice. Total rows stay the same, and no date and currency appears twice.

SECTION 05 · DISCUSSION

Five questions. Each one a design decision.

Every question sits on one part of the feed. Answer each against the mandates, not the product documentation.

WHERE EACH QUESTION LIVES



Q1

Warehouse or Lakehouse

This feed parses JSON with T-SQL in the warehouse. What would change if you built it in a Lakehouse with a PySpark notebook? Compare where parsing happens, the compute each one starts, and the latency for four rows a day.

Q2

Passing the payload

The Web Activity's response only exists at run time. How does the stored procedure activity receive it? Write the dynamic content expression and explain why the output is wrapped in `string()`.

Q3

Where to parse

Why shred the JSON inside the warehouse rather than reshape it in the pipeline first? Consider intermediate storage, how soon the data is queryable, and auditing the raw payload.

Q4

Reruns

Run the daily pipeline twice for the same date. What happens to `staging.CurrencyRates`, and where must the fix live when the warehouse does not enforce keys?

Q5

Weekends

Request rates for a Saturday. What date comes back in the payload, and what does that mean for a check that expects no missing dates?

THE FEED, ONCE IT RUNS

1.

API CALL PER DAY

4.

ROWS PER BUSINESS
DAY

~1,000.

ROWS PER YEAR

From copied web rates to a feed the risk desk can defend.

HOW YOU WILL BE EVALUATED

The three Done-when checks on page 06, plus five discussion answers framed against the mandates.

WHAT THIS PROVES

You can parse a nested API payload inside a relational warehouse. You can pass a Web Activity's output into a stored procedure. You can load history in one call and explain what makes a rerun safe.

INSTRUCTOR CONTACT

Surya Kumar · suryakumar01985@gmail.com